实　　验　　报　　告

学　　号	2108001 2078	姓　　名	姚嫣如	专业班级	网络空间安全 23
课程名称	**网络空间安全导论**			学期	**2025 秋**
任课教师	曲海鹏、李晓慧		完成日期		2025.11.05
小组成员	姚嫣如				

一、实验内容及步骤（55%）

（1）下载 IDA pro 绿化工具以及 LazyIDA 工具，并了解学习 IDA 使用过程

（2）使用 IDA 完成题目，找到 flag

二、实验结果（35%）

1、了解 IDA

（1）软件介绍

IDA，交互式反汇编器专业版 Interactive Disassembler Professional，是业界最成熟、先进的反汇编工具之一，是目前最好的一个静态反编译软件，IDA Pro 是一款交互式的，可编程的，可扩展的，多处理器的，交叉 Windows 或 Linux WinCE MacOS 平台主机来分析程序，已经成为事实上的分析敌意代码的标准并让其自身迅速成为攻击研究领域的重要工具。

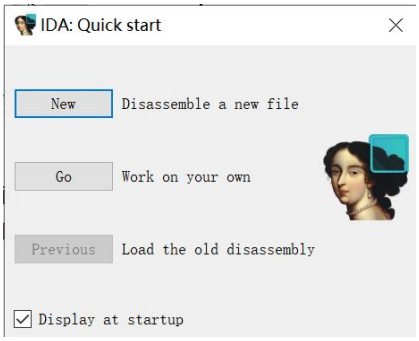
（2）优点

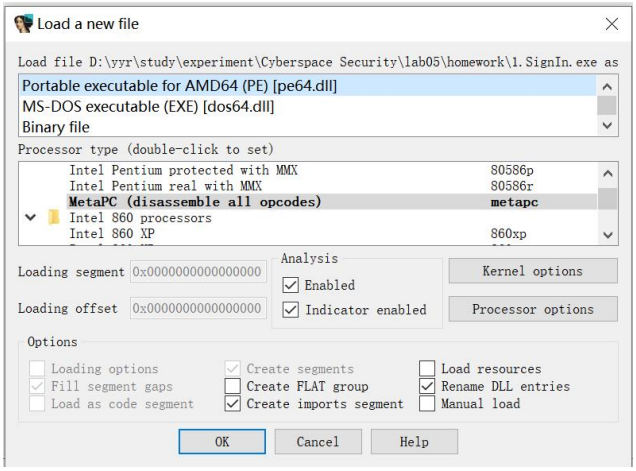
- 给函数、变量、数据结构等命名
- 更改数据表示（如数字、各种编码的字符串、数据结构）
- 搭建图标和代码流程图，简化对反汇编代码的理解
- 使用 C++中有关函数参数和结构定义类型信息，以便自动命名参数和变量
- 自动识别和命名汇编代码中的标准库函数

2、使用 IDA

（1）根据电脑配置打开 IDA Pro(64-bit).exe

选择 New 打开反编译新可执行文件（GO 直接进入主界面，Previous 加载已反编译的文件），将第一题拖入 IDA 中。





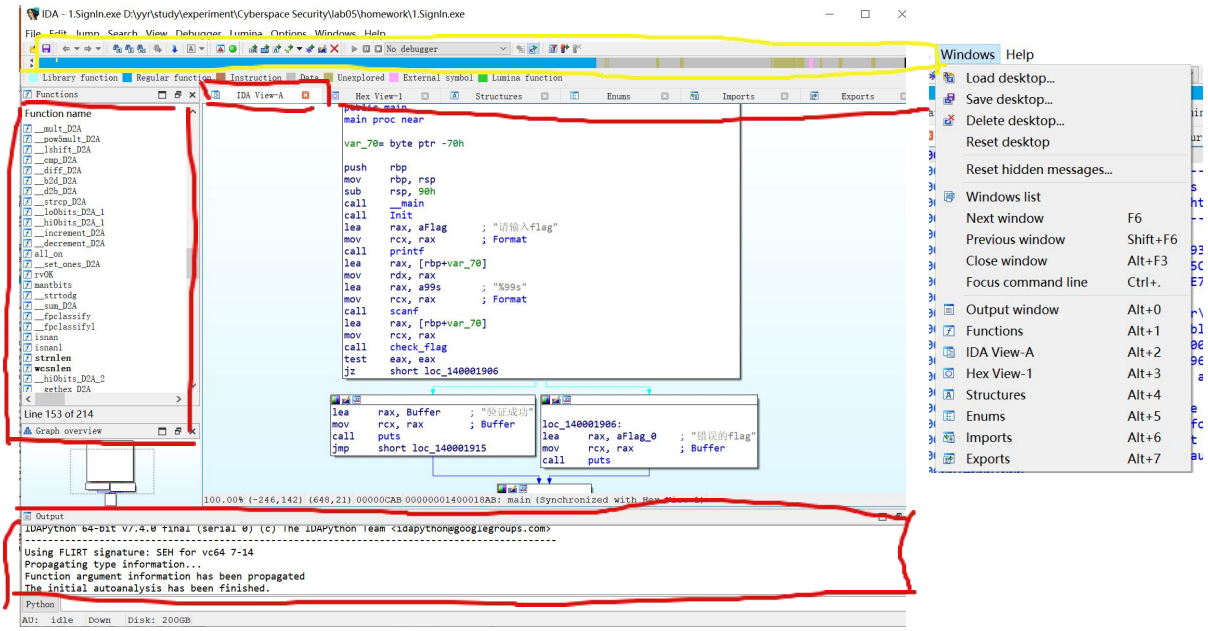
●使用 Binary file 加载文件，则需要手动输入加载段地址和相对偏移（Loading segent 和 Loading offset），IDA 不会自动分析代码，用户需根据具体需求自行反编译二进制代码，主要应用于分析动态保存的二进制代码、ShellCode 二进制代码分析

●Processor Type：可指定在反汇编过程中使用的处理器模块，多数情况，IDA 将从可执行文件的头中读取到信息，选择合适的处理器。

●kernel options：配置特定反汇编分析选项，IDA 可利用这些选项改进递归下降过程，通常 IDA 默认最优。

●processor options：选择适用于处理器模块的配置选项。

（2）选择文件后得到如下反汇编视图：



左边为 IDA 函数窗口。底部为输出窗口。反汇编视图有两种形式，图形和列表视图，可用空格切换。在工具栏 windows 中可任意打开数据窗口（反汇编、二进制、结构、函数、导入函数、导出函数）。

顶部彩色水平带为 IDA 导航栏，不同颜色代表不同类型的文件内容、代码、数据。

蓝色：常规指令函数，用户编写代码

黑色：间歇部分内容

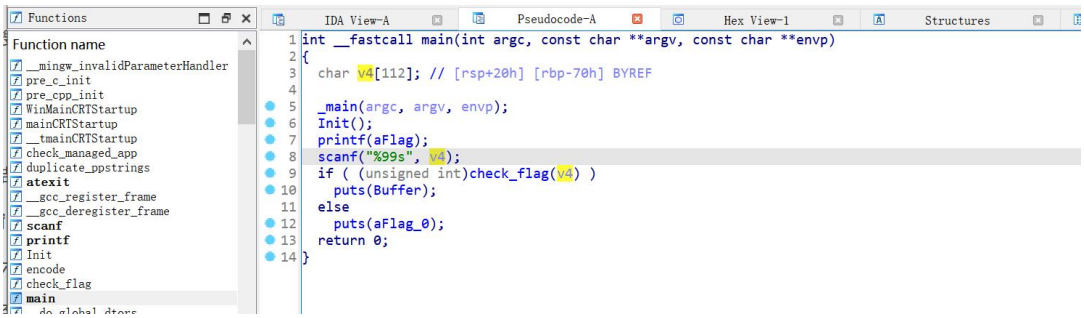
银白色：数据项部分内容

粉色：外部导入符号，如外部库函数

暗黄色：IDA 未识别的内容，需要用户根据需求自行分析。

3、homework——SignIn

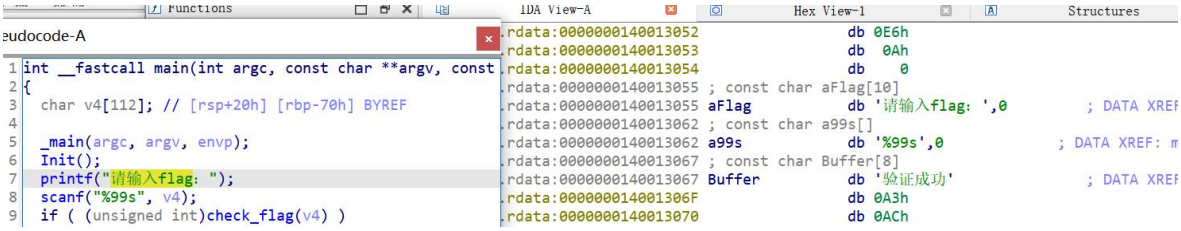
（1）在左侧函数窗口中找到 main 函数（可使用 Ctrl+F 搜索函数名），快捷键 F5 将函数逆向出来得到反编译窗口。



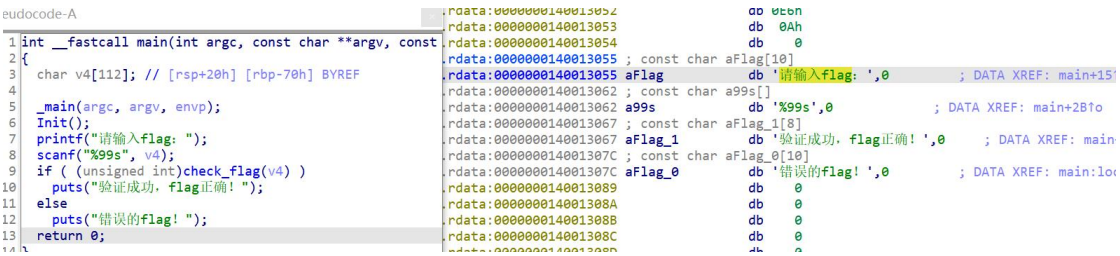
双击 aFlag，可以得到如下窗口，查看 aFlag 打印的字符串。



点击“请输入 flag”，使用快捷键 a，将选择的信息转化为字符串，之后回到程序界面，按 F5 刷新，程序处的“aFlag”就会变为“请输入 flag”



同理可将所有变量转化为字符串



（2）双击打开 check_flag ()，该函数用于检查输入的 flag 是否正确

```
_BOOL8 __fastcall check_flag(const char *a1)
{
  int v1; // ebx
  int v2; // eax
  char v4[8]; // [rsp+38h] [rbp-78h] BYREF
  char Buf1[128]; // [rsp+40h] [rbp-70h] BYREF

  strcpy(v4, "RealKey");
  v1 = strlen(a1);
  v2 = strlen(v4);
  encode((unsigned int)v4, v2, (_DWORD)a1, v1, (__int64)Buf1);
  return strlen(a1) == cipher_len && memcmp(Buf1, &cipher, cipher_len) == 0;
}
```

在函数 check_flag（）函数中，会通过 encode（）函数进行加密，比对用户输入的 flag 和答案密文 cipher 是否一致，并返回 bool 值。通过对变量名重命名（点击变量名，使用快捷键 n 进行重命名），使得程序逻辑更加清晰：

```
_BOOL8 __fastcall check_flag(const char *input)
{
  int input_len; // ebx
  int key_len; // eax
  char key[8]; // [rsp+38h] [rbp-78h] BYREF
  char Buf1[128]; // [rsp+40h] [rbp-70h] BYREF

  strcpy(key, "RealKey");
  input_len = strlen(input);
  key_len = strlen(key);
  encode((__int64)key, key_len, (__int64)input, input_len, (__int64)Buf1);
  return strlen(input) == cipher_len && memcmp(Buf1, &cipher, cipher_len) == 0;
}
```

可以注意到当前 encode 函数是无返回值的，双击进入 encode 函数，发现 IDA 错误的识别了该函数的返回值，可以使用 shift+delete 或者快捷键 v 将返回值删除

```
__int64 __fastcall encode(__int64 a1, int a2, __int64 a3, int a4, __int64 a5)
{
  __int64 result; // rax
  char v6; // [rsp+7h] [rbp-9h]
  signed int j; // [rsp+8h] [rbp-8h]
  int i; // [rsp+Ch] [rbp-4h]

  for ( i = 0; i < a2 / 2; ++i )
  {
    v6 = *(_BYTE *)(i + a1);
    *(_BYTE *)(i + a1) = *(_BYTE *)(6 - i + a1);
    *(_BYTE *)(a1 + 6 - i) = v6;
  }
  for ( j = 0; ; ++j )
  {
    result = (unsigned int)j;
    if ( j >= a4 )
      break;
    *(_BYTE *)(j + a5) = *(_BYTE *)(j % 7 + a1) ^ *(_BYTE *)(j + a3);
  }
  return result;
```

```
void __fastcall encode(__int64 key, int key_len, __int64 input, int input_len, __int64 Buf1)
{
  char v5; // [rsp+7h] [rbp-9h]
  int j; // [rsp+8h] [rbp-8h]
  int i; // [rsp+Ch] [rbp-4h]

  for ( i = 0; i < key_len / 2; ++i )
  {
    v5 = *(_BYTE *)(i + key);
    *(_BYTE *)(i + key) = *(_BYTE *)(6 - i + key);
    *(_BYTE *)(key + 6 - i) = v5;
  }
  for ( j = 0; j < input_len; ++j )
    *(_BYTE *)(j + Buf1) = *(_BYTE *)(j % 7 + key) ^ *(_BYTE *)(j + input);
}
```

通过观察该函数代码，可以发现 encode 函数传入时的 key 和 input 是字符串类型，但在 encode 中识别为了 int 类型，使得函数内容中需要大量使用强制类型转换，可以通过快捷键 y 将数据类型换为 char *类型，并重命名使得代码内容更直观清晰：

```
void __fastcall encode(char *key, int key_len, char *input, int input_len, char *Buf1)
{
  char v5; // [rsp+7h] [rbp-9h]
  int j; // [rsp+8h] [rbp-8h]
  int i; // [rsp+Ch] [rbp-4h]

  for ( i = 0; i < key_len / 2; ++i )
  {
    v5 = key[i];
    key[i] = key[6 - i];
    key[6 - i] = v5;
  }
  for ( j = 0; j < input_len; ++j )
    Buf1[j] = key[j % 7] ^ input[j];
}
```

至此，指针内存操作变为了简单易懂的数组操作，可以理解整个加密过程为将 key 进行翻转，并且让 key 模 7 的元素和 input 元素进行异或运算作为结果（相同为 0，不同为 1），由此逆向操作即可完成解密。

（3）拿到 key 和 cipher（作为 buf1 正确结果）

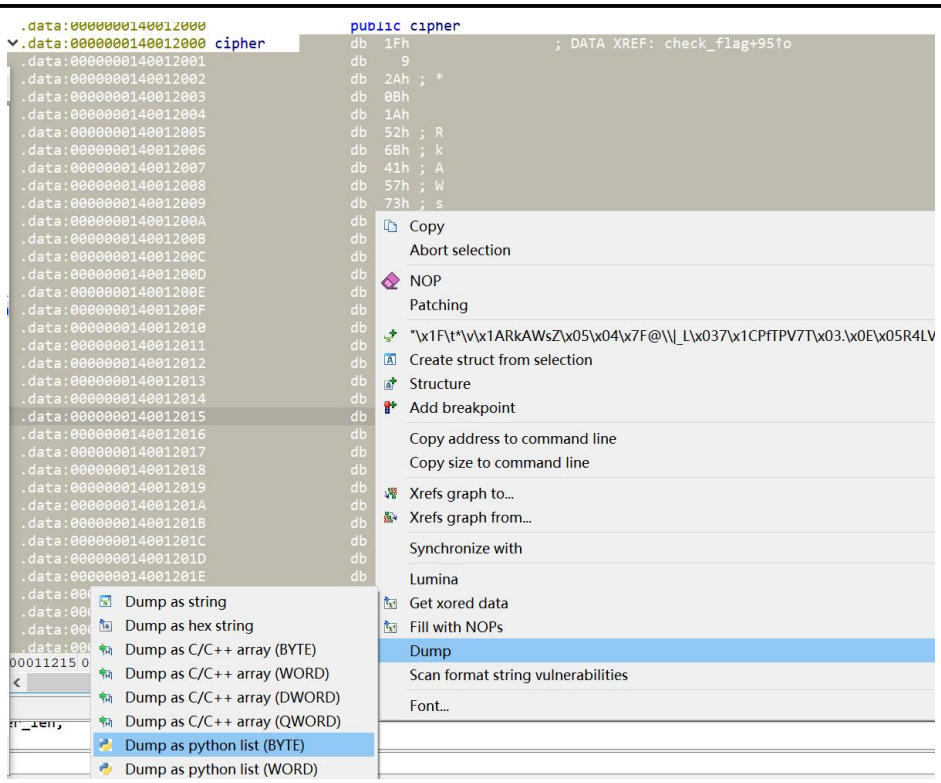
在 check_flag（）函数中，发现 key 值为“RealKey”

```
_BOOL8 __fastcall check_flag(const char *input)
{
  int input_len; // ebx
  int key_len; // eax
  char key[8]; // [rsp+38h] [rbp-78h] BYREF
  char Buf1[128]; // [rsp+40h] [rbp-70h] BYREF

  strcpy(key, "RealKey");
  input_len = strlen(input);
  key_len = strlen(key);
  encode((__int64)key, key_len, (__int64)input, input_len, (__int64)Buf1);
  return strlen(input) == cipher_len && memcmp(Buf1, &cipher, cipher_len) == 0;
}
```

双击打开 cipher，选中并向下多选中一个地址，右键选择 Dump—Dump as python list(BYTE)，在底下输出框中可以得到如下数据：

```
[+] Dump 0x140012000 - 0x14001202A (43 bytes) :
[0x1F, 0x09, 0x2A, 0x0B, 0x1A, 0x52, 0x6B, 0x41, 0x57, 0x73, 0x5A, 0x05, 0x04, 0x7F, 0x40, 0x5C, 0x7C, 0x5F, 0x4C, 0x03, 0x37, 0x1C, 0x50, 0x66, 0x54,
0x50, 0x56, 0x37, 0x54, 0x03, 0x2E, 0x0E, 0x05, 0x52, 0x34, 0x4C, 0x56, 0x78, 0x55, 0x58, 0x03, 0x2F, 0x00]
```



（4）编写 python 脚本，进行逆向解码，输出答案，得到 flag

```python
key = "RealKey"

cipher = [0x1F, 0x09, 0x2A, 0x0B, 0x1A, 0x52, 0x6B, 0x41, 0x57, 0x73, 0x5A, 0x05, 0x

# 翻转 key
key = key[::-1]
print(key)

output = bytearray()
for i in range(len(cipher)):
    output.append(cipher[i] ^ ord(key[i % len(key)]))

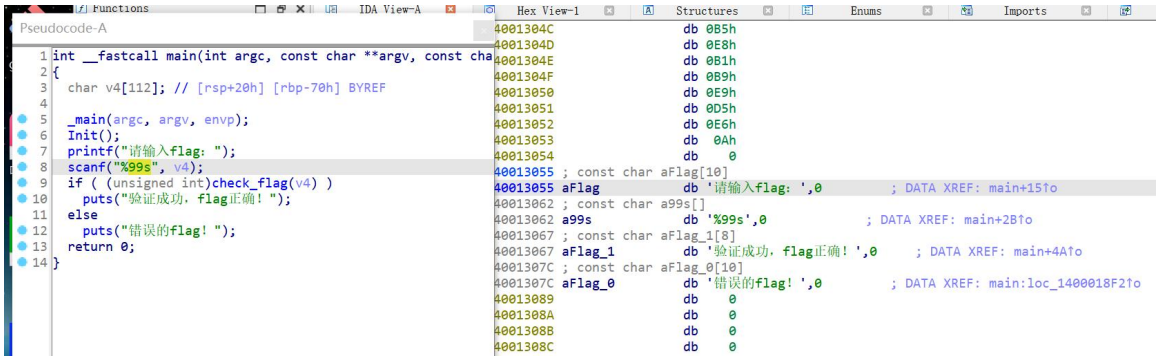
print(output)
```

```
bytearray(b'flag{798286da-9973-fee5-813e-febd7f53399f}')
```



4、homework——ezRE

（1）将题目 ezRE 拖入 IDA 中，得到反汇编视图，找到 main 函数双击，使用快捷键 F5 打开反编译窗口，并用快捷键 a 处理字符串：



```
Pseudocode-A
1  int __fastcall main(int argc, const char **argv, const cha
2  {
3    char v4[112]; // [rsp+20h] [rbp-70h] BYREF
4
5    _main(argc, argv, envp);
6    Init();
7    printf("请输入flag: ");
8    scanf("%99s", v4);
9    if ( (unsigned int)check_flag(v4) )
10     puts("验证成功，flag正确！");
11   else
12     puts("错误的flag！");
13   return 0;
14 }
```

```
4001304C                db 0B5h
4001304D                db 0E8h
4001304E                db 0B1h
4001304F                db 0B9h
40013050                db 0E9h
40013051                db 0D5h
40013052                db 0E6h
40013053                db  0Ah
40013054                db    0
40013055 ; const char aFlag[10]
40013055 aFlag          db '请输入flag: ',0          ; DATA XREF: main+15↑o
40013062 ; const char a99s[]
40013062 a99s           db '%99s',0             ; DATA XREF: main+2B↑o
40013067 ; const char aFlag_1[8]
40013067 aFlag_1        db '验证成功，flag正确！',0     ; DATA XREF: main+4A↑o
4001307C ; const char aFlag_0[10]
4001307C aFlag_0        db '错误的flag！',0          ; DATA XREF: main:loc_1400018F2↑o
40013089                db    0
4001308A                db    0
4001308B                db    0
4001308C                db    0
```

（3）双击 check_flag 函数，并进行重命名操作，理清代码逻辑，发现使用 encode 函数进行加密：

```
_BOOL8 __fastcall check_flag(const char *input)
{
  int input_len; // ebx
  int key_len; // eax
  char key[8]; // [rsp+38h] [rbp-78h] BYREF
  char Buf1[128]; // [rsp+40h] [rbp-70h] BYREF

  strcpy(key, "RealKey");
  input_len = strlen(input);
  key_len = strlen(key);
  encode((unsigned int)key, key_len, (_DWORD)input, input_len, (__int64)Buf1);
  return strlen(input) == cipher_len && memcmp(Buf1, cipher, cipher_len) == 0;
}
```

（4）进入 encode 函数，删除错误识别的返回类型，修改数据类型并重命名变量：

```
void __fastcall encode(char *key, int key_len, char *input, int input_len, char *Buf1)
{
  __int64 v5; // [rsp+0h] [rbp-10h]
  int j; // [rsp+8h] [rbp-8h]
  int i; // [rsp+Ch] [rbp-4h]

  v5 = 0x79654B656B6146i64;
  for ( i = 0; i < key_len; ++i )
    key[i] ^= *((_BYTE *)&v5 + i);
  for ( j = 0; j < input_len; ++j )
    Buf1[j] = key[j % 7] ^ input[j] ^ 0x12;
}
```

加密过程为循环 key 逐位于 v5 进行异或，再循环 key 模 7（循环利用 key）异或 input 异或十六进制数 0x12 作为结果

（5）拿到 key 和 cipher

```
7
8  strcpy(key, "RealKey");
9  input_len = strlen(input);
0  key_len = strlen(key);
1  encode((unsigned int)key, key_len, (_DWORD)input, input_len, (__int64)Buf1);
2  return strlen(input) == cipher_len && memcmp(Buf1, cipher, cipher_len) == 0;
3 }
```

双击 cipher，右键 Dump 得到 python list

```
.data:0000000140012000                  public cipher
.data:0000000140012000 cipher           db '`zy|iwp?" +&s?3!~y?*!g.5~&*w+!()"$#c#-z+*o',0
.data:0000000140012000                                          ; DATA XREF: check_flag+95↑o
.data:000000014001202B                  align 4
.data:000000014001202C                  public cipher_len
.data:000000014001202C cipher_len       dd 2Ah                  ; DATA XREF: check_flag+71↑r

[+] Dump 0x140012000 - 0x14001202C (45 bytes) :
[0x60, 0x7A, 0x79, 0x7C, 0x69, 0x77, 0x70, 0x3F, 0x22, 0x20, 0x2B, 0x26, 0x73, 0x3F, 0x33, 0x21, 0x7E, 0x79, 0x3F, 0x2A, 0x21, 0x67, 0x2E, 0x35, 0x7E,
0x26, 0x2A, 0x77, 0x2B, 0x21, 0x28, 0x29, 0x22, 0x24, 0x23, 0x63, 0x23, 0x2D, 0x7A, 0x2B, 0x2A, 0x6F, 0x00, 0x00, 0x2A]
```

（6）用 python 编写逆向解密脚本，得到 flag

```python
key1 = "RealKey"

cipher = [0x60, 0x7A, 0x79, 0x7C, 0x69, 0x77, 0x70, 0x3F, 0x22, 0x20, 0x2B, 0x26, 0

v5 = 0x79654B656B6146
v5 = v5.to_bytes( length: 7, byteorder: 'little')

key = []
for i in range(len(key1)):
    key.append(ord(key1[i]) ^ v5[i])

flag = []
for i in range(len(cipher)):
    flag.append(cipher[i] ^ key[i % 7] ^ 0x12)

for x in flag:
    print(chr(x), end='')
```

```
D:\yyr\study\python\pythonProject\.venv\Scripts\python.exe "D:\yyr\study\experiment\Cyb
flag{eb94804a-57fb-83a8-e48e-702061e55a98}
进程已结束，退出代码为 0
```

```
========================
欢迎来到CTF逆向入门指南课程!
Author: Real返璞归真
========================
请输入flag: flag{eb94804a-57fb-83a8-e48e-702061e55a98}
```

4、2025_sd_pyb_re2

（1）用 IDA 打开文件，在函数窗口找到 main 函数，双击点开后重新命名变量，替换字符串

```
__int64 __fastcall main(int a1, char **a2, char **a3)
{
  char mem[288]; // [rsp+0h] [rbp-190h] BYREF
  char input[104]; // [rsp+120h] [rbp-70h] BYREF
  unsigned __int64 v6; // [rsp+188h] [rbp-8h]

  v6 = __readfsqword(0x28u);
  Init(mem, a2, a3);
  if ( (unsigned int)my_gets(input, 100) )
  {
    putchar(10);
    if ( (unsigned int)check(mem, input) )
      puts("✅ 验证成功！");
    else
      puts("❌ 验证失败！");
    return 0LL;
  }
  else
  {
    puts("输入无效，程序退出");
    return 1LL;
  }
}
```

（2）依次查看各个函数作用，更改误识别的数据类型

```
void __fastcall Init(char *mem)
{
  __int64 v1; // rbx
  int src[40]; // [rsp+10h] [rbp-E0h] BYREF
  __int64 v3; // [rsp+C0h] [rbp-30h]
  __int64 v4; // [rsp+C8h] [rbp-28h]
  int v5; // [rsp+D0h] [rbp-20h]
  __int16 v6; // [rsp+D4h] [rbp-1Ch]
  unsigned __int64 v7; // [rsp+D8h] [rbp-18h]

  v7 = __readfsqword(0x28u);
  src[0] = 3;
  src[1] = 5;
  src[2] = 1;
  src[3] = 6;
  src[4] = 2;
  src[5] = 4;
  src[6] = 5;
  src[7] = 6;
  src[8] = 3;
  src[9] = 1;
  src[10] = 2;
  src[11] = 5;
  src[12] = 4;
  src[13] = 6;
  src[14] = 1;
  src[15] = 3;
  src[16] = 2;
  src[17] = 5;
  src[18] = 4;
```

```
void __fastcall sub_17D3(char *input, int a2)
{
  size_t input_len; // [rsp+18h] [rbp-8h]

  printf("请输入flag: ");
  if ( fgets(input, a2, stdin) )
  {
    input_len = strlen(input);
    if ( input_len )
    {
      if ( input[input_len - 1] == 10 )
        input[input_len - 1] = 0;
    }
  }
  else
  {
    puts("读取输入失败");
  }
}
```

```
__int64 __fastcall change(char flag_i, unsigned int mem)
{
  __int64 result; // rax

  switch ( mem )
  {
    case 1u:
      result = (unsigned __int8)flag_i ^ 5u;
      break;
    case 2u:
      result = (unsigned int)(unsigned __int8)flag_i + 1;
      break;
    case 3u:
      result = (unsigned int)(unsigned __int8)flag_i - 1;
      break;
    case 4u:
      result = rol_and_ror(flag_i);
      break;
    case 5u:
      result = rol((unsigned __int8)flag_i, 1LL);
      break;
    case 6u:
      result = ror((unsigned __int8)flag_i, 1LL);
      break;
    default:
      printf("未知opcode: %d\n", mem);
      result = (unsigned __int8)flag_i;
      break;
  }
  return result;
}
```

```
_BOOL8 __fastcall check(char *mem, char *input)
{
  unsigned __int8 flag_i; // [rsp+19h] [rbp-17h]
  int i; // [rsp+1Ch] [rbp-14h]
  int check_flag; // [rsp+20h] [rbp-10h]
  int j; // [rsp+24h] [rbp-Ch]
  int round; // [rsp+28h] [rbp-8h]

  if ( !(unsigned int)check_len(input) )
    return 0LL;
  for ( i = 0; i <= 37; ++i )
    mem[i] = input[i];
  memcpy(mem + 38, mem, 0x26uLL);
  check_flag = 1;
  for ( j = 0; j <= 37; ++j )
  {
    flag_i = mem[j];
    for ( round = 0; round <= 39; ++round )
      flag_i = change(flag_i, *(unsigned int *)&mem[4 * round +
    mem[j + 38] = flag_i;
    if ( flag_i != mem[j + 76] )
    {
      check_flag = 0;
      return check_flag != 0;
    }
  }
  return check_flag != 0;
}
```

（3）找到 key 和密文

通过研究 check 函数的逻辑，将*(unsigned int *)&mem[4 * round + 116]作为加密规则，而 mem+116 存储的是 src 的副本，因此 key 实际值就是 src 数组的 40 个元素，可以在 Init 中找到 key 的定义：

key=[3,5,1,6,2,4,5,6,3,1,2,5,4,6,1,3,2,5,4,6,1,3,2,5,4,6,1,3,5,2,4,6,1,3,2,5,4,6,1,3]

在 check 中最后加密后的输入需要与 mem[j+76]比对，因此可知 mem+76 往后的部分就是密文，也可以在 Init 中找到相关定义：

// mem + 76 开始的 8 字节（QWORD）

*(_QWORD *)(mem + 76) = 0x687AADC5A19FB6B4LL;

// mem + 84 开始的 8 字节（QWORD）

*(_QWORD *)(mem + 84) = 0x7B70681D707BAD77LL;

// mem + 92 开始的 8 字节（QWORD）

*(_QWORD *)(mem + 92) = 0x777BAE1D7CA07076LL; // v3 的值

// mem + 100 开始的 8 字节（QWORD）

*(_QWORD *)(mem + 100) = 0xF968A068B4AE7CB4LL; // v4 的值

// mem + 108 开始的 4 字节（DWORD）

*((_DWORD *)mem + 27) = 2003874678; // v5 的值（mem+108 = 4*27 = 108）

// mem + 112 开始的 2 字节（WORD）

*((_WORD *)mem + 56) = -17761; // v6 的值（mem+112 = 2*56 = 112）

从代码中可以看到使用的是多字节数据，用来表示比字节更大的数值，将多个字节子盒成多字节数据。

类型	全称	字节数	二进制位数	范围（无符号）
WORD	双字节	2	16	0 ~ 65535
DWORD	双字（四字节）	4	32	0 ~ 4294967295
QWORD	四字（八字节）	8	64	0 ~ 18446744073709551615

而多字节数据在内存中存储时，会面临字节的排列顺序问题，在逆向分析中一般使用小端序存储，即低字节低地址，高字节高地址，即将十六进制值按每 2 位一组拆分（一字节两位十六进制），从左到右对应高字节到低字节，则 mem+76=0x687AADC5A19FB6B4LL 拆分后为 0x68、0x7A、0xAD、0xC5、0xA1、0x9F、0xB6、0xB4，之后按照小端序逆序排列：0xB4, 0xB6, 0x9F, 0xA1, 0xC5, 0xAD, 0x7A, 0x68

enc = [0xB4, 0xB6, 0x9F, 0xA1, 0xC5, 0xAD, 0x7A, 0x68, 0x77, 0xAD, 0x7B, 0x70, 0x1D, 0x68, 0x70, 0x7B, 0x76, 0x70, 0xA0, 0x7C, 0x1D, 0xAE, 0x7B, 0x77, 0xB4, 0x7C, 0xAE, 0xB4, 0x68, 0xA0, 0x68, 0xF9, 0x76, 0xB3, 0x70, 0x77, 0x9F, 0xBA]

Initial 函数在给 mem 数组赋值时用的时多字节类型，但在 check 中比较的时单字节（mem[j+76]），如果不按小端序转换，提取到的密文字节顺序会错误，导致解码失败，出现乱码。

（4）编写逆向解码程序，得到 flag

```python
# 右移函数
def rorr(a1, a2):  3 用法
    a2 %= 8
    return ((a1 >> a2) | (a1 << (8 - a2))) & 0xFF # & 0xFF 防止因运算产生超过8位的数值
# 左移函数
def roll(a1, a2):
    a2 %= 8
    return ((a1 << a2) | (a1 >> (8 - a2))) & 0xFF

# 正向：a1 - (a1+3)左移2 - (结果-2)右移2 - 结果-1 - 输出
def rol_and_ror(a1):
    v1 = roll((a1 + 3) & 0xFF,  a2: 2)
    return (rorr((v1 - 2) & 0xFF,  a2: 2) - 1) & 0xFF

# 逆向：输入 - +1 - 左移2 - +2 - 右移2 - -3 - 原始a1
def rol_and_ror_rev(a1):  1 个用法
    step1 = (a1 + 1) & 0xFF
    step2 = roll(step1,  a2: 2)
    step3 = (step2 + 2) & 0xFF
    step4 = rorr(step3,  a2: 2)
    return (step4 - 3) & 0xFF
```

```python
# change
def reverse_change(flag_i, mem):  1 个用法
    if mem == 1:
        return flag_i ^ 5
    elif mem == 2:
        return (flag_i - 1) & 0xFF
    elif mem == 3:
        return (flag_i + 1) & 0xFF
    elif mem == 4:
        return rol_and_ror_rev(flag_i)
    elif mem == 5:
        return rorr(flag_i,  a2: 1)
    elif mem == 6:
        return roll(flag_i,  a2: 1)
    else:
        return flag_i

# 正确的密文（按小端序解析原始16进制值，共38字节）
enc = [
    0xB4, 0xB6, 0x9F, 0xA1, 0xC5, 0xAD, 0x7A, 0x68,  # 0x687AADC5A19FB6B4（小端）
    0x77, 0xAD, 0x7B, 0x70, 0x1D, 0x68, 0x70, 0x7B,  # 0x7B70681D707BAD77（小端）
    0x76, 0x70, 0xA0, 0x7C, 0x1D, 0xAE, 0x7B, 0x77,  # 0x777BAE1D7CA07076（小端）
    0xB4, 0x7C, 0xAE, 0xB4, 0x68, 0xA0, 0x68, 0xF9,  # 0xF968A068B4AE7CB4（小端）
    0x76, 0xB3, 0x70, 0x77,                          # 0x7770B376（v5=2003874678的小端）
    0x9F, 0xBA                                       # 0xBA9F（v6=-17761的小端，0xBA9F为补码）
]

key = [3,5,1,6,2,4,5,6,3,1,2,5,4,6,1,3,2,5,4,6,1,3,5,2,4,6,1,3,2,5,4,6,1,3]
```

```python
# 解密过程
plaintext = []
for c in enc:
    current = c
    # 逆序执行所有操作
    for op in reversed(key):
        current = reverse_change(current, op)
    plaintext.append(chr(current))

print("解密结果:", ''.join(plaintext))
```

```
解密结果:  flag{c603c78508728b45d73f4df0b012e83a}
```

三、心得总结（写出在完成实验过程中遇到的问题、解决方法，以及体会、收获等）（10%）

通过本次实验，学习到了如何使用 IDA 进行静态分析：重命名变量、类型修复、删除返回值等，并使用常用的快捷键；如何找到关键的 key 和 cipher，并提取 cipher 中的数据；分析加密函数逻辑，并使用 python 编写逆向解码程序。

出现的问题：

1、最开始密文未按照小端序转换，导致最后 flag 的输出未乱码

```
D:\yyr\study\python\pythonProject\.venv\Scripts\python.exe "D:\yyr\study\experiment\C
ÜT⍰èQT⍰*§è*⍰B*="§¤⍰Qþ"¤þè=èïB.*QmÝ
```